



MyScoutee Registry

CONTROLLED OPERATIONS DOCUMENTATION

REGISTRY OPERATIONS

MyScoutee Registry Operator Manual

Trust model, real-registry deployment, secure operation, local administrative rails, recovery, and controlled distribution for MyScoutee Registry 1.0.0.

DOCUMENT ID MSC-ROM-001	DOCUMENT VERSION 1.0.0
APPLICABLE SOFTWARE MyScoutee Registry 1.0.0	STATUS ● Controlled draft
CLASSIFICATION Internal / Controlled partner distribution	RELEASE DATE 2026-07-30
LANGUAGE EN	CANONICAL FORMAT Self-contained HTML

Document Control

Document status and applicability

Document owner	MyScoutee Registry project — registry operations and protocol stewardship
Intended audience	Registry infrastructure operators, security officers, registry administrators, release managers, recovery personnel, and controlled regional partners
Normative current deployment surface	<code>compose.production.yaml</code> for a source-backed deployment, or the build-free two-image stack under <code>tools/docker/offline/</code> ; both keep the Go registry and Nginx edge as separate services
Review triggers	Any change to the protocol, Compose topology, image build, SQLite migrations, keys, CLI rails, backup format, Java integration, or distribution policy

Version History

Version	Date	Status	Change
1.0.0	2026-07-30	Controlled draft	Initial combined operator manual for the implemented v1 registry and its open production-distribution boundaries.

Version and Evidence Boundary

This manual describes the repository state reviewed for version 1.0.0. Passing source tests is not equivalent to completing the external production drills. A new image, migration, topology, or partner bundle requires a release-specific review and recorded qualification evidence.

Evidence Base

The implemented statements in this manual are derived from the following version-controlled sources:

- `README.md` , the internal qualification and protocol build inputs under `guides/manuals/protocols/` , and the standalone `MyScoutee_Registry_Protocol_Developer_Manual_v1.0.0_EN.pdf` ;
- `compose.production.yaml` , `compose.yaml` , `compose.provisioned-key.yaml` , and `Dockerfile` ;
- `tools/docker/registry-edge.Dockerfile` , the release-candidate files under `tools/docker/offline/` , `tools/package-offline-stack.sh` , and `tools/offline-stack-contract-test.sh` ;
- `.env.production.example` and `tools/docker/registry.conf.template` ;
- the Go implementation under `cmd/registry/` and `internal/` , including embedded SQLite migrations;
- the current MyScoutee Java registry client, persistence, outbox, Operator API, and frontend integration in the sibling backend repository.

IMPLEMENTED means present in current source. **EXPLORE/DEMO** means real code running against isolated seeded demo state. **PROPOSED** is recommended design work, not a shipped artifact. **OPEN GAP** identifies a control or qualification boundary that is not closed by the current repository.

Table of Contents

Purpose, Scope, and Safety Language
.....

Implemented, Explore, and Proposed Surfaces
.....

Architecture and Trust Model
.....

System Requirements and Planning Baseline
.....

Real Registry Installation
.....

Configuration, Scope, Keys, and SQLite
.....

DNS, TLS, Firewalls, and Time
.....

Lifecycle, Health, Monitoring, and Checkpoints
.....

Registry-Local Administrative CLI
.....

Operational Protocol Rules and API Boundary
.....

Backup and Restore
.....

Upgrade and Rollback
.....

Image, Offline Bundle, and Distribution Policy
.....

Restricted and Mainland-China Operation
.....

Security Operations and Incident Response
.....

Retirement and Removal
.....

Known Gaps and Limitations
.....

Quick Reference
.....

1. Purpose, Scope, and Safety Language

This manual explains how to deploy and operate one independently governed MyScoutee Registry 1.0.0 instance. It combines the minimum protocol trust model needed by an operator with concrete production Compose procedures and registry-local administrative controls.

In Scope

- a single-host Linux deployment using Docker Engine and Docker Compose v2;
- the separate Go registry and Nginx services in `compose.production.yaml`;
- the immutable registry scope, Ed25519 registry identity, VOPRF keyring, and SQLite state;
- public HTTPS, health, monitoring, local administrative CLI rails, backup, restore, upgrades, incidents, and removal;
- controlled regional or offline delivery where public source and image registries cannot be assumed reachable.

Out of Scope

- a multi-node or active-active registry, automatic failover, cross-registry replication, or federation;
- an authenticated public image-release service, signed OCI supply chain, OS package/systemd installer, or unattended registry updater; the working-tree offline packager is implemented, but it is not a turnkey supported public installer;
- legal, tax, beneficial-owner, payment, payout, or acquisition decisions;
- a guaranteed SLA, RTO, RPO, throughput, user count, or storage-retention limit;
- automatic repair of inconsistent append-only data.

Do Not Convert Evidence Rails into Payment Authority

Settlements, exit reviews, ownership transfers, and final exit allocations are signed technical evidence. The implementation does not execute money movement, store bank instructions, create invoices, decide tax treatment, or replace external legal approval.

2. Implemented, Explore, and Proposed Surfaces

Surface	Status	Operational meaning
Real registry	IMPLEMENTED	<code>compose.production.yaml</code> ; ordinary registry startup, mandatory non-demo scope, persistent production volume, no seed command, and a separate TLS Nginx edge.
Explore registry	EXPLORE/DEMO	<code>compose.yaml</code> ; explicit <code>start-demo</code> , <code>REGISTRY_DEMO_SEED=true</code> , a <code>demo:</code> scope, demo-specific filenames, deterministic signed fixtures, and loopback publication by default.

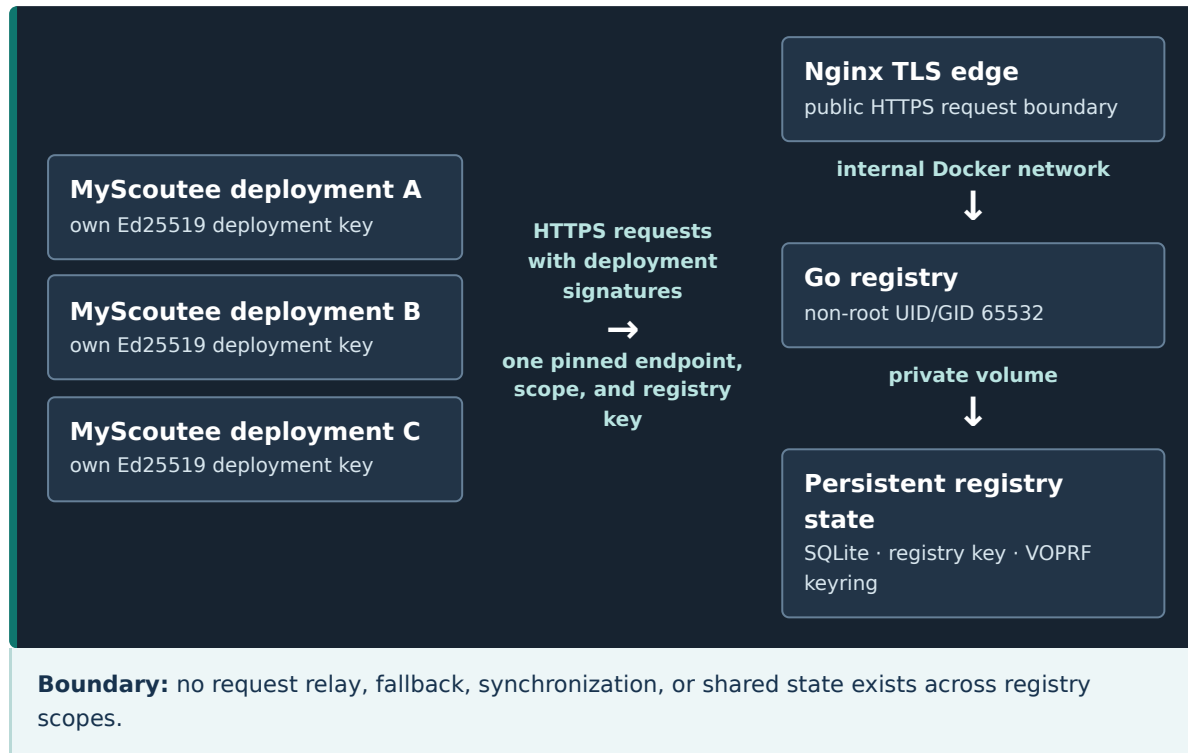
Surface	Status	Operational meaning
Go runtime image	IMPLEMENTED	A stripped, statically linked binary in a scratch image, UID/GID 65532:65532 , no shell or package manager, port 8080 metadata, and no production secret embedded.
Nginx edge	IMPLEMENTED	A separate first-party edge image derived from <code>nginx:1.28-alpine</code> . Both the source Compose deployment and offline stack use the image with the reviewed configuration template embedded; host TLS files remain runtime bind mounts.
Two first-party release images	IMPLEMENTED IN WORKING TREE	The Go image remains separate. <code>tools/docker/registry-edge.Dockerfile</code> builds <code>myscoutee-registry-edge:<version>-prod</code> and embeds the reviewed template. Published digest/signature policy remains open.
Offline deployment bundle	PACKAGER IMPLEMENTED AUTHENTICITY EXTERNAL	<code>tools/package-offline-stack.sh</code> creates a reproducible Linux/amd64 two-image archive with a pull-free/no-build Compose file, manifest, environment template, audit copy of the Nginx template, and checksums. It does not create a vendor signature or SBOM.
<code>1.0.0-full-prod</code> combined image	NOT IMPLEMENTED	Not recommended as the default. Combining Nginx and Go requires a process supervisor and still does not contain DNS, TLS keys, persistent state, policy, or orchestration.

Image Role Is Determined by Command, Configuration, and State

The same Go binary can run an ordinary real registry or the guarded Explore seed path. Never infer “real” or “demo” from the image tag alone. Real and demo instances must have different scopes, keys, databases, volumes, endpoints, backups, and MyScoutee client state.

3. Architecture and Trust Model

One sovereign registry serving multiple independently signed MyScoutee deployments



Actors and Controls

Actor	Trusted capability	What that capability does not prove
Registry infrastructure operator	Host, container, network, TLS, volume, backup, and availability control.	Truth of a deployment's reported QMAU, revenue, company claim, or legal evidence.
Registry administrator	Access to the local CLI and the registry's existing state/key paths; can append reviews and other controlled records.	A separate human signature. CLI actor and reference fields are audit labels, not remote identity proof.
MyScoutee deployment	Possession of its own Ed25519 deployment private key; signs requests and preserves signed receipts.	That its aggregate claim is honest or that one local account equals one human.
Authenticated MyScoutee Operator	Selects a registry, initiates registration, submits company data, and approves local product updates.	Authority to mutate the registry's local manager-only review rails.
Release authority	Separately held Ed25519 package-signing key identified by <code>pkey_...</code> .	Registry identity. The registry key <code>rkey_...</code> must never be accepted as an implicit package key.

Cryptographic Boundaries

- The registry has one Ed25519 identity. Its deterministic key ID is `rkey_` plus the first 32 hexadecimal characters of the SHA-256 digest of the complete Ed25519 SPKI DER public key.
- A deployment's assigned `dep_...` code is public. Control is proven by the corresponding private key, not by secrecy of the code.
- Protocol signatures cover canonical LF-delimited messages, not incidental JSON field order. Binary fields use padded standard base64; SHA-256 values use lowercase `sha256:` form.
- The linear hash-linked ledger is authoritative. Compact RFC 9162-style Merkle proofs and signed daily checkpoints make a ledger prefix independently auditable but do not prove claim truth.
- Registry TLS authenticates the network endpoint. The self-signed registry identity then binds protocol scope and signing key. Both checks are required.

One Registry, One Scope, One Active Binding

Every registry instance has one mandatory immutable `REGISTRY_SCOPE`. Each MyScoutee deployment communicates directly with one exact registry endpoint and scope. The registry does not forward or replicate a signed request to another scope. A deployment must complete signed deactivation and preserve its old receipts before binding to another registry.

An organization may operate more than one independent registry—for example one inside a restricted national network and another for a global population—but a single MyScoutee deployment does not use them as active/standby endpoints. Separate registries require separate domains, keys, databases, volumes, backups, and deployment populations.

Trust Is Tamper Evidence, Not Oracle Truth

QMAU and revenue requests are deployment-signed aggregate claims. The registry can prove who submitted unchanged bytes, preserve corrections, and sign acceptance; it cannot inspect the deployment's private activity or payment evidence and cannot independently establish truthfulness.

4. System Requirements and Planning Baseline

Required Platform Capabilities

Area	Requirement	Verification
Host	A supported Linux Docker host with a maintained kernel and enough privileges to manage Docker, firewall, time, storage, and protected files.	<code>docker info</code>
Container runtime	Docker Engine and Docker Compose v2. Source builds additionally require access to the Go 1.25 builder image and modules, unless all layers are preloaded.	<code>docker compose version</code>

Area	Requirement	Verification
Storage	Durable local storage with working file locks, atomic filesystem operations, directory sync, and SQLite WAL/fsync semantics. Do not place the volume on unqualified network storage.	Release-specific power-loss and restore drill
Time	Reliable UTC synchronization. Signed requests default to a five-minute skew window, and ledger/checkpoint guards reject clock rollback.	<code>timedatectl status</code> or the platform equivalent
Network	Stable DNS, a certificate-valid HTTPS origin, inbound client reachability, and a management path separated from public traffic.	External TLS, DNS, and firewall tests
Backup	Encrypted backup storage able to preserve the complete registry state unit and a clean-host restore environment.	Recorded restore drill

Suggested Starting Sizes

The repository defines no qualified hardware minimum or throughput guarantee. The following figures are planning estimates, not support limits:

Use	vCPU	Memory	Durable SSD	Use condition
Evaluation or small controlled registry	2	2 GiB	20 GiB	Low request volume, monitored growth, no production SLA.
Initial production planning point	4	4–8 GiB	50–100 GiB	Retain at least 25% free space and resize from measured ledger, nonce, announcement, claim, and identity-presence growth.

SQLite is intentionally configured with one serialized connection. State is append-only and durable replay nonces are retained. Full integrity checks occur at bootstrap and health boundaries. Capacity therefore depends on history size, health frequency, presence submissions, deployment count, and traffic shape—not only current active users.

Architecture Qualification

The source Dockerfile sets `G00S=linux` but takes the target architecture from the build environment. The implemented offline bundle contract deliberately accepts only `linux/amd64`; there is no published multi-architecture compatibility matrix. Record and qualify the exact host architecture, kernel, Docker version, filesystem, image digest, and source commit.

5. Real Registry Installation

Do Not Start Production with the Repository-Root `compose.yaml`

The repository-root `compose.yaml` is the seeded Explore/demo surface. A source-backed real registry uses `compose.production.yaml`, ordinary startup, a non-demo scope, and no `REGISTRY_DEMO_SEED` guard. The separately packaged offline bundle also contains a file named `compose.yaml`; that reviewed bundle file is a real, build-free deployment surface.

Global Compose Command Rule

Source-backed examples throughout this manual run from the reviewed repository release and use `compose.production.yaml`; a source build may use `--build`. Offline examples run from the extracted, authenticated bundle and substitute its `compose.yaml`; never add `--build` or permit a pull, and retain `--pull never` when creating or recreating services. Commands that require `compose.provisioned-key.yaml` have no current offline-bundle equivalent. Installation and Compose examples assume an approved root administrative shell, because `/opt/myscoutee-registry` is mode 0750 and the protected environment is mode 0600; do not loosen those modes to make an unprivileged shell work.

5.1 Prepare the Release Directory and Protected Files

For a source-backed installation, populate `/opt/myscoutee-registry` with the exact reviewed repository release, then run the source commands in this chapter with that directory as the current working directory. It must contain `compose.production.yaml`, the Docker build context, and `tools/docker/registry.conf.template`. Source transfer and authentication are release-authority responsibilities; this repository does not contain a fetch installer. For an offline installation, extract an authenticated controlled bundle to a dedicated directory and follow §5.2 from the extracted bundle root.

```
sudo install -d -m 0750 /opt/myscoutee-registry
sudo install -d -m 0700 /opt/myscoutee-registry/tls

sudo install -o root -g root -m 0600 /secure/source/fullchain.pem \
/opt/myscoutee-registry/tls/fullchain.pem
sudo install -o root -g root -m 0600 /secure/source/privkey.pem \
/opt/myscoutee-registry/tls/privkey.pem

cd /opt/myscoutee-registry

sudo install -o root -g root -m 0600 .env.production.example \
/etc/myscoutee-registry.env
sudoedit /etc/myscoutee-registry.env
```

In the protected environment file, set at least:

```
MYSCOUTEE_VERSION=1.0.0
REGISTRY_PRODUCTION_IMAGE=myscoutee-registry:1.0.0-prod
REGISTRY_SCOPE=example:region-a
REGISTRY_PUBLIC_HOSTNAME=registry.example.com
```

```

REGISTRY_PUBLIC_BIND_ADDRESS=0.0.0.0
REGISTRY_HTTP_PORT=80
REGISTRY_HTTPS_PORT=443
REGISTRY_TLS_CERTIFICATE_PATH=/opt/myscoutee-registry/tls/fullchain.pem
REGISTRY_TLS_PRIVATE_KEY_PATH=/opt/myscoutee-registry/tls/privkey.pem

```

5.2 Offline-Bundle Installation

The release-candidate packager produces `dist/myscoutee-registry-stack_1.0.0_linux_amd64.tar.gz`. After authenticating the outer archive through the approved vendor/release channel, extract it on the target and verify its internal contract:

```

sha256sum --check SHA256SUMS

docker image load --input images/myscoutee-registry.tar
docker image load --input images/myscoutee-registry-edge.tar

docker image inspect myscoutee-registry:1.0.0-prod \
  --format '{{.Id}} {{.Os}}/{{.Architecture}}'
docker image inspect myscoutee-registry-edge:1.0.0-prod \
  --format '{{.Id}} {{.Os}}/{{.Architecture}}'

```

Compare both reported image IDs with `IMAGE-MANIFEST.json`. The internal `SHA256SUMS` file detects corruption after the outer bundle is trusted; it is not publisher authentication. The current packager does not generate a detached vendor signature.

The offline bundle excludes TLS material, production environment, registry/VOPRF keys, and SQLite state. Before starting Compose, install the TLS pair at the host paths in §5.1, then copy and edit the environment:

```

sudo install -d -m 0700 /opt/myscoutee-registry/tls
sudo install -o root -g root -m 0600 /secure/source/fullchain.pem \
  /opt/myscoutee-registry/tls/fullchain.pem
sudo install -o root -g root -m 0600 /secure/source/privkey.pem \
  /opt/myscoutee-registry/tls/privkey.pem

sudo install -o root -g root -m 0600 registry.env.example \
  /etc/myscoutee-registry.env
sudoedit /etc/myscoutee-registry.env

docker compose \
  --env-file /etc/myscoutee-registry.env \
  -f compose.yaml \
  config --quiet

docker compose \
  --env-file /etc/myscoutee-registry.env \
  -f compose.yaml \
  up --pull never -d

```

The bundled Compose file has no `build` section and sets `pull_policy: never` for both services. It keeps the read-only filesystems, capability restrictions, `no-new-privileges`, health checks, and internal backend network. The target needs Docker Engine and Docker Compose but no source checkout, Go toolchain, GitHub, Docker Hub, or outbound build access.

Its current Compose surface uses generated first-run registry identity. A provisioned external registry signing key requires the separately reviewed repository overlay or a future bundle overlay; do not improvise an unreviewed mount.

5.3 Choose the Registry Identity Mode

Generated first-run key

Keep `REGISTRY_GENERATE_SIGNING_KEY=true` only for the first pristine start. The registry atomically creates a mode-0600 Ed25519 PKCS#8 key in the production data volume.

Provisioned key

Set `REGISTRY_PROVISIONED_KEY_PATH`, use `compose.provisioned-key.yaml`, and run the explicit one-shot `initialize` command before ordinary startup.

Generated-Key First Start

```
docker compose \
  --env-file /etc/myscoutee-registry.env \
  -f compose.production.yaml \
  config --quiet

docker compose \
  --env-file /etc/myscoutee-registry.env \
  -f compose.production.yaml \
  up --build -d
```

Provisioned-Key First Start

The external key must be an Ed25519 PKCS#8 PEM, a regular non-symlink file, mode `0600`, and readable as numeric container UID/GID `65532:65532`.

```
sudo chown +65532:+65532 -- /secure/registry-signing-key.pem
sudo chmod 0600 -- /secure/registry-signing-key.pem

docker compose \
  --env-file /etc/myscoutee-registry.env \
  -f compose.production.yaml \
  -f compose.provisioned-key.yaml \
  run --rm --build registry initialize

docker compose \
  --env-file /etc/myscoutee-registry.env \
  -f compose.production.yaml \
  -f compose.provisioned-key.yaml \
  up -d
```

Initialization Is Deliberately Fail-Closed

Ordinary startup will not pair an existing signing key with a pristine database. It will not generate a missing key for an established database. Restore the matching state instead of forcing a new identity. Only `initialize` may establish a pristine database under a provisioned key.

5.4 Commission and Record the Identity

```
docker compose \
  --env-file /etc/myscoutee-registry.env \
  -f compose.production.yaml \
  ps

docker compose \
  --env-file /etc/myscoutee-registry.env \
  -f compose.production.yaml \
  logs --tail 100 registry nginx

curl --fail --show-error \
  https://registry.example.com/healthz

curl --fail --show-error \
  https://registry.example.com/v1/registry/identity

curl --head --fail --show-error \
  http://registry.example.com/
```

The HTTPS checks must succeed without `-k` or another insecure-TLS option. The HTTP request should return a permanent 308 redirect to the configured HTTPS origin. Record the following in the commissioning evidence:

- ☐ source commit and Go image ID/digest;
- ☐ the exact Nginx image ID/digest;
- ☐ `registry_scope`, `registry_key_id`, and the complete identity response;
- ☐ public-key SPKI SHA-256 fingerprint and the independently verified TLS certificate chain;
- ☐ initial ledger index, entry count, and head hash;
- ☐ Compose configuration, host/kernel/filesystem versions, firewall rules, and backup target;
- ☐ an external request from an allowed MyScoutee network and a denied request from a disallowed management source.

After the first verified start, set both generation switches to `false` in the protected environment file. Recreate the registry container so Compose applies the changed environment; a plain `restart` does not do so. This turns accidental volume/key loss into a startup failure instead of a new identity:

```
# Set these values in /etc/myscoutee-registry.env:
REGISTRY_GENERATE_SIGNING_KEY=false
REGISTRY_GENERATE_GLOBAL_IDENTITY_VOPRF_KEY=false

docker compose \
  --env-file /etc/myscoutee-registry.env \
  -f compose.production.yaml \
  up -d --force-recreate registry
```

5.5 Do Not Enrol Clients Until Commissioning Is Complete

Publish the intended HTTPS origin, scope, registry key ID, and full SPKI fingerprint over an independent trusted channel. MyScoutee deployment operators need those values to distinguish the intended registry from a different validly operating registry at a compromised endpoint.

6. Configuration, Scope, Keys, and SQLite

6.1 Registry Runtime Variables

Variable	Default/current bound	Operator rule
REGISTRY_LIST EN_ADDR	:8080	Keep the Go service on the internal Docker network. Do not publish 8080 on the production host.
REGISTRY_SCOPE	Required; 3-128 lowercase ASCII characters from a-z, 0-9, ., :, _, -	Choose once. The first character must be a letter or digit. A changed scope against an existing database fails startup.
REGISTRY_DATA BASE_PATH	/data/registry.db	SQLite database in the persistent production volume.
REGISTRY_SIGNING_KEY_PATH	/data/registry-signing-key.pem	Registry Ed25519 identity. Keep with the matching database or use the provisioned-key overlay.
REGISTRY_GENERATE_SIGNING_KEY	true	Permit only on a deliberate pristine first start; set false after commissioning.
REGISTRY_GLOBAL_IDENTITY_VOPRF_KEYRING_PATH	/data/registry-global-identity-voprf-keyring.json	Sensitive versioned RFC 9497 VOPRF seed keyring; back up with SQLite and the registry signing key.
REGISTRY_GENERATE_GLOBAL_IDENTITY_VOPRF_KEY	true	Generate the first keyring only; set false after commissioning.
REGISTRY_DEMO_SEED	false	Never enable for a real registry. It is accepted only with start-demo.
REGISTRY_TIME STAMP_SKEW	5m; must be positive	Increase only after a threat and retry review; fix clocks rather than widening casually.
REGISTRY_MAX_REQUEST_BODY_BYTES	65536; valid 1024-1048576	Coordinate with the edge body limit and global-identity chunk sizing.
REGISTRY_VALUATION_MULTIPLIER_BASIS_POINTS	30000; valid 1000-100000	Technical, non-binding settlement input. A change can create a new immutable settlement revision.

Variable	Default/current bound	Operator rule
REGISTRY_CHECKPOINT_INTERVAL	1m	How often the worker attempts completed UTC-day finalization.
REGISTRY_SHUTDOWN_TIMEOUT	10s binary default; not forwarded by either shipped Compose file	The shipped deployments therefore use the fixed default. A different value requires a reviewed Compose environment addition and a coordinated, tested change to the 15-second Compose stop grace period.
REGISTRY_HEALTHCHECK_URL	http://127.0.0.1:8080/healthz	Used by the scratch image's built-in healthcheck command.

6.2 Edge Variables

Variable	Current default	Operational effect
REGISTRY_PUBLIC_HOSTNAME	Required	DNS server name, redirect authority, Nginx <code>server_name</code> , and TLS SNI name.
REGISTRY_PUBLIC_BIND_ADDRESS	Required; example 0.0.0.0	Host interface used for port publication. Prefer a specific address when exposure must be restricted.
REGISTRY_HTTP_PORT / REGISTRY_HTTPS_PORT	80 / 443	Custom ports are supported; clients must use the exact final HTTPS origin.
REGISTRY_EDGE_MAX_BODY_SIZE	64k	Keep no larger than the backend limit unless the pair is deliberately qualified.
REGISTRY_EDGE_REQUEST_RATE	10r/s	Per-address Nginx request-rate zone.
REGISTRY_EDGE_REQUEST_BURST	20	Nginx burst accepted without delay before HTTP 429.
REGISTRY_EDGE_CONNECTION_LIMIT	20	Concurrent connections per observed address. Review NAT concentration before changing.

6.3 SQLite Durability and Migrations

The implementation uses the CGo-free `modernc.org/sqlite` driver with:

- WAL journal mode, foreign keys enabled, `synchronous=FULL`, and a 5-second busy timeout;
- one open/idle connection and serialized transactional writes;
- embedded numbered schema migrations, each applied and recorded in its own transaction during open;
- append-only triggers plus comprehensive identity, signature, ledger, direct-row, Merkle, and checkpoint verification.

Direct query rows are written in the same transaction as their signed source. They are not eventually consistent projections and are not rebuilt or silently repaired. A failed integrity audit makes affected operations fail closed.

6.4 Key Lifecycle Summary

Key	Rotation	Loss/compromise effect
Registry Ed25519 signing key	No supported in-place identity rotation in v1.	Loss stops verification and startup. Compromise affects the registry identity and requires a controlled new-registry migration, not a silent file replacement.
Global-identity VOPRF keyring	<code>rotate-global-identity-key</code> appends a version; old versions remain for explicit correction. Restart is required.	Loss prevents future evaluation/correction and must fail closed. Possession together with restricted commitments enables offline dictionary testing of likely low-entropy identifiers.
TLS private key	Operator-owned certificate renewal/replacement at Nginx.	Does not change the registry signing identity, but can permit endpoint impersonation while compromised.
MyScoutee package- signing key	Outside this registry's key lifecycle; separately pinned by each deployment.	Used to verify a Debian artifact offered in an update manifest. Never store its private key in the registry.

7. DNS, TLS, Firewalls, and Time

7.1 DNS

- Create an A record for the exact public or private address used by allowed MyScoutee deployments. Publish an AAAA record only after IPv6 bind and firewall behavior are verified.
- Use a stable hostname covered by the certificate. Do not use an origin path, credentials, query, or fragment in a MyScoutee registry base URL.
- Keep DNS ownership and recovery credentials outside the registry host. A DNS takeover can redirect clients even though registry-key comparison should detect a changed registry identity.
- When changing address, keep the same hostname and registry state. Current MyScoutee binding rules also pin the endpoint; changing the origin is not a transparent failover operation.

7.2 TLS

Nginx terminates TLS 1.2/1.3, disables session tickets, adds one-year HSTS and `X-Content-Type-Options: nosniff`, and redirects HTTP to the configured HTTPS hostname and port. The Go service remains plain HTTP only on the internal Docker network.

Replace certificate and private-key files as a matched protected pair, then recreate or reload Nginx and verify both its container health and the external certificate:

```
docker compose \
  --env-file /etc/myscoutee-registry.env \
  -f compose.production.yaml \
  up -d --force-recreate nginx

openssl s_client \
  -connect registry.example.com:443 \
  -servername registry.example.com \
  < /dev/null

curl --fail --show-error \
  https://registry.example.com/healthz
```

The current repository does not issue or renew production certificates. Automate renewal only with a separately reviewed host process and alert on expiry.

7.3 Firewall

Flow	Default port	Rule
MyScoutee deployments → Nginx HTTPS	TCP 443	Allow only the intended networks where practical. If deployments share NAT, account for per-address edge limits.
HTTP redirect	TCP 80	Optional but currently configured. It performs only a 308 redirect; clients should store the final HTTPS origin.

Flow	Default port	Rule
Management SSH	Operator-defined	Allow only through VPN, bastion, or fixed administrative sources. SSH is a host prerequisite, not installed by this registry.
Go backend	Container TCP 8080	Internal Docker network only. Never publish directly on the production host.
Outbound Internet	None required by the running Go protocol	Images, build modules, external monitoring, DNS, and certificate renewal may need separately controlled egress. Pre-stage them for restricted operation.

7.4 Time

Monitor clock offset and NTP state. Registry writes cannot move before registry creation, the current ledger head, or a finalized day. Deployment requests outside the configured skew are rejected. If time is wrong, stop writes, correct the host clock through the approved time service, verify state, and resume; never rewrite timestamps in SQLite.

8. Lifecycle, Health, Monitoring, and Checkpoints

Source and Offline Compose Filenames

The global command rule in §5 applies here: examples use the source-backed `compose.production.yaml`; an installed offline bundle substitutes its `compose.yaml`, never builds or pulls, and retains `--pull never` on service creation or recreation. The service names and protected environment file remain the same.

8.1 Common Lifecycle Commands

```
docker compose \
  --env-file /etc/myscoutee-registry.env \
  -f compose.production.yaml \
  up -d
```

```
docker compose \
  --env-file /etc/myscoutee-registry.env \
  -f compose.production.yaml \
  ps
```

```
docker compose \
  --env-file /etc/myscoutee-registry.env \
  -f compose.production.yaml \
  logs --tail 200 registry nginx
```

```
docker compose \
  --env-file /etc/myscoutee-registry.env \
  -f compose.production.yaml \
  restart registry
```

```
docker compose \
```

```
--env-file /etc/myscoutee-registry.env \
-f compose.production.yaml \
down
```

`down` without `-v` removes containers and networks but preserves the named production data volume. Never add `-v` during routine lifecycle work.

8.2 Health Semantics

`GET /healthz` returns HTTP 200 only after storage and integrity verification. Its JSON includes protocol version, scope, registry key ID, ledger index, entry count, and ledger head hash. The Go image's `/registry healthcheck` command calls that endpoint with a three-second timeout. Nginx has a separate TLS healthcheck and starts only after the registry becomes healthy.

```
curl --fail --show-error \
https://registry.example.com/healthz

docker inspect \
--format '{{json .State.Health}}' \
myscoutee-registry-registry-1
```

The container name depends on the Compose project name; resolve it with `docker compose ps` instead of assuming the example.

Health Is Not a Cheap Ping

A health request performs integrity work. The current Compose services probe every 10 seconds, and public `/healthz` is also reachable through Nginx. Do not add high-frequency external probes. Measure health latency as history grows and qualify any interval or endpoint split before changing the release topology.

8.3 Monitoring Minimum

Signal	Alert condition	Response
Container state/restarts	Unhealthy, restart loop, unexpected exit, or Nginx unable to reach registry.	Preserve logs and check key, volume, migration, TLS, and integrity errors.
External HTTPS	Certificate failure, identity endpoint failure, wrong scope/key ID, or sustained latency.	Compare against commissioning evidence before accepting traffic.
Disk and inode use	Less than the locally defined headroom; this manual recommends 25% free as a starting threshold.	Expand storage. Do not delete append-only rows or nonce history.
Backup	Missed job, failed encryption/upload, or restore evidence older than policy.	Run and verify a new complete recovery-unit backup.
Clock	NTP unsynchronized or offset approaches request-skew policy.	Correct the time source before accepting signed writes.

Signal	Alert condition	Response
HTTP status	Rising 429, 5xx, signature failures, replay conflicts, or request-size failures.	Separate abuse, NAT concentration, client defects, outage retry, and integrity faults.
Ledger/checkpoints	Unexpected head regression, missing completed-day checkpoint, or proof failure.	Stop writes and follow the integrity-incident procedure.

The Go service writes structured JSON logs to stdout. No Prometheus metrics endpoint, distributed trace exporter, or registry-specific log shipper is implemented. Configure Docker/host log rotation and a protected external collector without recording private CLI output.

8.4 Daily Checkpoints and Merkle Witnessing

The background worker attempts completed UTC-day checkpoint finalization at the configured interval. A checkpoint never represents the still-open current UTC day. Retain external copies of selected identity responses, signed checkpoints, and Merkle tree heads so a later consistency proof can show append-only growth.

9. Registry-Local Administrative CLI

Run production CLI commands inside the existing registry container so they use the same protected environment, signing key, VOPRF keyring, scope, and SQLite volume:

```
docker compose \
  --env-file /etc/myscoutee-registry.env \
  -f compose.production.yaml \
  exec -T registry \
  /registry COMMAND [OPTIONS]
```

Operational commands use an existing-state bootstrap. A missing or mistyped database, scope, key, or identity fails rather than creating new state. Exact idempotent duplicates exit successfully and return the original signed result. Invalid flags, stale targets, integrity failures, storage failures, and idempotency conflicts exit non-zero.

CLI Access Is Administrative Authority

Restrict host and Docker access. Registry signatures prove what this authority recorded; they do not authenticate a separate reviewer. Commands such as `show-operator-claim`, `settlements`, and `exit-allocation` detail can expose protected company/contact or beneficiary data. Do not pipe them into shared logs or tickets.

9.1 Complete Implemented Command Inventory

Rail	Commands	Purpose and boundary
Runtime	<code>healthcheck</code>	Calls the configured health endpoint. It does not create or repair state.
Initialization	<code>initialize</code>	One-shot identity initialization under a provisioned key. Not a routine start command.
Explore only	<code>start-demo</code>	Guarded deterministic demo seed and ordinary server. Forbidden for a real registry.
Announcements	<code>publish-announcement</code>	Strict local append of a GENERAL, UPDATE, MAINTENANCE, or SECURITY announcement. There is no public publish API.
Company claim review	<code>list-operator-claims</code> , <code>show-operator-claim</code> , <code>approve-operator-claim</code> , <code>reject-operator-claim</code>	Review-safe list, private detail, and exact-current-claim approval/rejection. Stale target fields fail closed.

Rail	Commands	Purpose and boundary
Claim eligibility	<code>suspend-operator-claim</code> , <code>reinstate-operator-claim</code>	Independent eligibility state for an approved claim. It does not rewrite verification, QMAU, identity, or history.
Registry cases	<code>list-registry-cases</code> , <code>show-registry-case</code> , <code>flag-registry-case</code> , <code>clear-registry-case</code>	Append-only local case/evidence rail for a typed subject. It is not a public support-ticket API.
Exit review	<code>freeze-exit-review</code> , <code>decide-exit-review</code> , <code>dispute-exit-review</code> , <code>withdraw-exit-review</code> , <code>list-exit-reviews</code> , <code>show-exit-review</code>	Freezes exact checkpoint, claim, group, QMAU/revenue, and settlement boundaries; records buyer/auditor decisions. No ownership or payment transfer.
Ownership transfer	<code>prepare-ownership-transfer</code> , <code>decide-ownership-transfer</code> , <code>cancel-ownership-transfer</code> , <code>complete-ownership-transfer</code> , <code>list-ownership-transfers</code> , <code>show-ownership-transfer</code>	Two-phase, registry-signed effective group-membership change pinned to an eligible verified exit. No payment execution.
Final exit allocation	<code>create-exit-allocation</code> , <code>verify-exit-allocation</code> , <code>list-exit-allocations</code> , <code>show-exit-allocation</code>	Copies exact frozen settlement revisions and conserves amounts per currency. Contractual evidence only.
Accounting reads	<code>leaderboard</code> , <code>revenue</code> , <code>settlements</code> , <code>global-identity-dedup</code>	Signed or protected local reads. Revenue and settlement commands can expose private amounts.
Settlement calculation	<code>calculate-settlement</code>	Creates or returns an immutable technical monthly settlement revision for one completed month and currency.
Global identity key	<code>rotate-global-identity-key</code>	Appends a new VOPRF key version, verifies metadata, and requires a registry restart.
Merkle proofs	<code>merkle-proof</code> , <code>merkle-consistency</code> , <code>verify-merkle-proof</code> , <code>verify-merkle-consistency</code>	Generate or locally verify signed inclusion and append-only consistency evidence.

9.2 Representative Controlled Procedures

Publish a Reviewed Announcement

```
docker compose \
  --env-file /etc/myscoutee-registry.env \
  -f compose.production.yaml \
```

```
exec -T registry \
/registry publish-announcement --file - \
< /secure/reviewed-announcement.json
```

Take a consistent backup before publication. The input is one strict UTF-8 JSON object, maximum 256 KiB, with no unknown or duplicate fields. Publication is append-only. Revocation or supersession is another announcement, not an edit.

Review a Company Claim

```
docker compose --env-file /etc/myscoutee-registry.env \
-f compose.production.yaml exec -T registry \
/registry list-operator-claims --status PENDING_REVIEW --limit 50

docker compose --env-file /etc/myscoutee-registry.env \
-f compose.production.yaml exec -T registry \
/registry show-operator-claim --deployment-id dep_0123456789abcdef0123456789abcdef

docker compose --env-file /etc/myscoutee-registry.env \
-f compose.production.yaml exec -T registry \
/registry approve-operator-claim \
--deployment-id dep_0123456789abcdef0123456789abcdef \
--claim-action-id opa_0123456789abcdef0123456789abcdef \
--group-id opg_0123456789abcdef0123456789abcdef \
--legal-name 'Example Cooperative' \
--reviewer-id network-review-team \
--review-reference case:2026-0042 \
--idempotency-key approve-example-2026-0042
```

Copy deployment, claim action, group, and legal name from the same private `show` result. Use bounded non-personal actor/reference fields. Re-run `show` immediately before the decision when review takes time.

Calculate and Inspect a Technical Settlement

```
docker compose --env-file /etc/myscoutee-registry.env \
-f compose.production.yaml exec -T registry \
/registry calculate-settlement --period 2026-06 --currency EUR

docker compose --env-file /etc/myscoutee-registry.env \
-f compose.production.yaml exec -T registry \
/registry settlements --period 2026-06 --currency EUR --limit 20
```

The calculation accepts a completed UTC month and one supported currency. It computes the 5% pool once after aggregate commission-basis revenue, uses the six-complete-month QMAU boundary for share weights, and never converts currencies. It remains non-binding technical evidence.

Rotate the VOPRF Key

```
docker compose --env-file /etc/myscoutee-registry.env \
-f compose.production.yaml exec -T registry \
/registry rotate-global-identity-key

docker compose --env-file /etc/myscoutee-registry.env \
-f compose.production.yaml restart registry
```

Back up the complete recovery unit immediately before and after rotation. Older versions remain in the keyring to support explicit corrections; rotation is not deletion of prior capability.

9.3 Command Help Is the Flag Authority

For every rail, run `/registry COMMAND --help` in the container and copy cursor, event hash, claim action, group, review, transfer, allocation, and idempotency values exactly. Do not infer identifiers or modify opaque cursors.

10. Operational Protocol Rules and API Boundary

10.1 Public API Groups

Purpose	Endpoints	Trust boundary
Identity and registration	GET <code>/v1/registry/identity</code> POST <code>/v1/deployments/register</code>	Read-only self-signed identity, then deployment proof-of-possession registration.
QMAU and revenue	POST <code>/v1/mau/batches</code> POST <code>/v1/revenue/batches</code> receipt GETs	Deployment-signed aggregate snapshots and immutable correction revisions. No raw user/payment identifiers.
Operator network	POST <code>/v1/operator/actions</code> claim status and leaderboard GETs	Deployment mutations; registry-signed status and snapshot-bound cursor reads.
Announcements and updates	GET <code>/v1/announcements</code>	Registry-signed pull feed. No remote command, push, public writer, download, or automatic install.
Global identity	VOPRF key/evaluate, link/action, presence batch, and dedup snapshot endpoints	RFC 9497 P256-SHA256 VOPRF and opaque commitments; privacy-enhancing but not anonymous PSI.
Settlement history	POST <code>/v1/settlements/query</code>	Deployment-signed private query returning only exact historical beneficiary memberships for that deployment.
Audit evidence	Checkpoint, Merkle inclusion, and Merkle consistency GETs	Registry-signed append-only evidence over ledger prefixes.
Health	GET <code>/healthz</code>	Public operational integrity result; no secrets, but not a lightweight liveness-only check.

10.2 Strict HTTP Rules

- Protocol POST bodies are UTF-8 `application/json`, exactly one JSON object, within the configured body bound, with no duplicate or unknown fields.

- Signed mutation paths reject query strings. All endpoints reject percent-encoded path aliases. Nginx intentionally forwards the original target without rewriting it.
- RFC 3339 timestamps must be UTC and end in literal `Z`. Periods and dates use their documented canonical forms.
- The current Java REAL client requires an HTTPS origin and does not follow redirects. Insecure HTTP is limited to explicitly allowed demo/development hosts.
- Current Java defaults are a 5-second connect timeout, 20-second request timeout, and 1 MiB maximum response.

10.3 Retry and Idempotency

Every signed mutation uses a timestamp, nonce, idempotency key, payload hash, and signature. Apply these rules after a timeout or lost response:

1. Preserve the original normalized payload and idempotency key.
2. Create a fresh timestamp, nonce, and signature for the retry.
3. An exact accepted retry returns the stored signed result without another append.
4. The same signer/idempotency key with a different payload hash is a conflict.
5. Reusing an accepted nonce for a different request is a replay conflict.
6. Back off on transport errors, HTTP 429, and retryable 5xx responses. Do not blindly retry validation, signature, stale-revision, or idempotency-conflict failures.

Durable nonce retention intentionally grows. Do not prune it outside a future versioned retention protocol.

10.4 QMAU, Revenue, and Global Identity

- `qmau-v1` counts a distinct local non-admin/non-demo/non-test human account with at least two substantive actions on at least two days in the rolling 30-day window. The registry receives only the aggregate and evidence commitment.
- Leaderboard measured weight is the arithmetic mean of active latest QMAU revisions for the six most recent complete UTC months; missing months contribute zero. This is deployment-level QMAU, not globally deduplicated people.
- Revenue is daily aggregate net captured revenue by supported ISO-4217 currency, never a transaction list. Currencies are never converted or combined.
- Exact global dedup requires consent-bound VOPRF links and qualified-presence submissions matching an accepted QMAU revision. Unlinked QMAU remains counted. Dedup measurement does not redistribute deployment weight.
- The current Java sender uses 512 commitments per chunk under the default body limit; with the Go bounds of 4096 chunks, the current interoperable ceiling is 2,097,152 commitments per submission. This is an ingress bound, not a supported network-size guarantee.

10.5 Announcement and Package Signatures

An UPDATE announcement is an offer for a MyScoutee operator to inspect, not installation consent. The registry signs the announcement and manifest with `rkey_...`. The referenced Debian artifact has a separate detached Ed25519 signature from `pkey_...`. A deployment must verify HTTPS origin, exact byte size, SHA-256 digest, compatibility range, package key ID, and detached package signature before any privileged local install workflow is approved.

11. Backup and Restore

11.1 Recovery Unit

Back up these items together under one evidence ID:

- ☐ the complete production data volume, including SQLite database, WAL/SHM files if present, generated registry signing key, and VOPRF keyring;
- ☐ or, for a provisioned signing key, the complete volume plus the exact external key under its separate custody control;
- ☐ the protected production environment file;
- ☐ Compose files, Nginx template, source commit, Go/Nginx image IDs and digests;
- ☐ TLS certificate chain and private key under the certificate policy;
- ☐ commissioning identity, latest health head, selected checkpoints/Merkle evidence, and backup manifest/checksums.

Raw Copy Requires Quiescence

The current repository has no registry backup command and the scratch image has no shell or SQLite CLI. Do not copy a changing database, WAL, and SHM sequentially. Stop/quiesce the service, use SQLite's online backup API through a separately qualified tool, or take an atomic storage snapshot.

11.2 Quiesced Backup Procedure

Complete stopped-volume backup

- 1 Record current identity, ledger index/head, checkpoint state, image IDs, Compose status, and UTC time.
- 2 Stop public ingress, then run `docker compose ... down` without `-v`.
- 3 Resolve the exact Docker volume carrying logical volume `registry-production-data`. Confirm its Compose project labels; never guess a similarly named volume.
- 4 Take an atomic storage snapshot or a root-controlled archive that preserves numeric owner `65532`, permissions, timestamps, and every file in the volume.
- 5 Back up external key/TLS/environment/release evidence separately under the same backup ID.
- 6 Encrypt, checksum, and store the backup according to local retention and geographic policy.
- 7 Restart the exact release, verify health and identity, and record downtime.

For a Docker local volume, use `docker volume inspect` to resolve its mountpoint, then operate only on that exact resolved directory while the services are stopped. Storage drivers and managed platforms may require a provider snapshot instead of direct host access.

11.3 Restore Procedure

Clean-host restore

- 1 Isolate the target from MyScoutee traffic and install the exact recorded Docker/Compose release surface.
- 2 Create a new empty target volume. Keep generation switches false.
- 3 Restore every volume file with numeric UID/GID and modes preserved. Restore the exact external signing key if provisioned.
- 4 Restore the protected environment with the original scope and key paths. Never substitute another scope's key, database, or VOPRF keyring.
- 5 Start only the registry service and inspect logs. Startup must verify identity, migrations, append-only triggers, signatures, direct rows, ledger, Merkle index, and checkpoints.
- 6 Verify `/healthz` and `/v1/registry/identity` against the backup manifest, then verify selected Merkle/checkpoint evidence.
- 7 Start/recreate Nginx, test normal TLS externally, and re-enable allowed traffic gradually.

A restore is successful only when scope, registry key ID, SPKI fingerprint, ledger index/head, and selected signed evidence match the recovery record. A generated replacement key or empty healthy ledger is not recovery.

11.4 Restore Testing

Test restore to a clean isolated host at the frequency required by the local RPO/RTO. Record duration, image and source IDs, migration behavior, filesystem, integrity result, key ID, ledger head, and external proof verification. The repository's tests include temporary closed-database restore evidence, but production storage/TLS/host restoration remains an external release drill.

12. Upgrade and Rollback

No Registry Auto-Updater

The registry does not poll for or install its own image updates. Its announcement feed concerns MyScoutee deployment packages. Compose changes and registry image replacement are explicit registry-operator actions.

Controlled registry upgrade

- 1 Review release notes, protocol compatibility, embedded migrations, image contents, source commit, Nginx changes, and all open qualification evidence.
- 2 Build or receive the candidate under a new immutable version tag. Record the content digest; do not overwrite the current release tag as the only rollback reference.
- 3 Run `go test ./...`, `go test -race ./...`, the tagged E2E suite, `go vet ./...`, image inspection, and the Java-to-Go rail qualification against the candidate.
- 4 Take and verify a complete pre-upgrade recovery-unit backup. Rehearse its restore on an isolated host.
- 5 Schedule downtime or drain public ingress. Stop the current services cleanly.
- 6 Load/build the candidate images and update only the intended image/version references. Validate the fully resolved Compose configuration.
- 7 Start the registry. Embedded migrations run transactionally during database open; startup then performs full integrity verification.
- 8 Start Nginx and verify TLS, health, identity, unchanged scope/key ID, non-regressed ledger head, registration/retry behavior, representative CLI reads, checkpoints, and Merkle proofs.
- 9 Retain the pre-upgrade image and backup until the observation window closes.

Current source-built deployment uses:

```
docker compose \
  --env-file /etc/myscoutee-registry.env \
  -f compose.production.yaml \
  config --quiet

docker compose \
  --env-file /etc/myscoutee-registry.env \
  -f compose.production.yaml \
  up --build -d
```

Rollback Is Backup Restore, Not Just an Older Container

Migrations are forward-applied at open, and no general downgrade compatibility contract is implemented. If rollback is required after the candidate touched the database, restore the matching pre-upgrade database/key/keyring unit and prior images. Do not simply start an older binary against migrated state unless that exact downgrade path was qualified.

13. Image, Offline Bundle, and Distribution Policy

13.1 Is Publishing the Registry Image Unsafe?

Version 1.0.0 distribution boundary: the real registry images and offline bundle are not publicly distributed as a supported release. The repository contains a working-tree release-candidate packager, while publisher authentication, release qualification, support policy, and delivery remain controlled external responsibilities.

The current final image contains the compiled Go binary and empty runtime directories, not production SQLite data, registry/VOPRF/TLS private keys, environment files, or announcements. Publishing that exact image does not by itself disclose deployed secrets. It does expose the binary and protocol implementation for analysis and creates support, vulnerability-response, licensing, provenance, and patch-distribution obligations.

Security must not depend on keeping the image secret. Nevertheless, until an OCI signing, SBOM, immutable-tag, disclosure, and support policy is in place, controlled partner distribution is the lower-risk operational choice. Use an access-controlled regional registry or signed offline media and still assume recipients can inspect the binary.

13.2 Implemented Two-Image Release-Candidate Surface

IMPLEMENTED IN WORKING TREE The offline contract keeps two independently replaceable artifacts:

1. `myscoutee-registry:1.0.0-prod` — the existing non-root scratch Go runtime;
2. `myscoutee-registry-edge:1.0.0-prod` — an Nginx 1.28 Alpine-derived edge with the exact reviewed template embedded, still receiving certificate/key files as runtime mounts.

The build input should pin the approved Nginx base by digest through `NGINX_BASE_IMAGE`. The generated manifest records exact image IDs, while the bundled Compose uses versioned references and `pull_policy: never`. A published online surface should additionally pin both by digest. Keeping the services separate preserves their privilege, logs, health, and update boundaries. A combined `1.0.0-full-prod` image would need a process supervisor and would still require orchestration for ports, volumes, networks, health, restart policy, TLS mounts, and environment. It therefore does not solve the distribution problem.

13.3 Implemented Offline Bundle Contract

IMPLEMENTED IN WORKING TREE Build the two exact local Linux/amd64 images, then package them:

```
docker build -t myscoutee-registry:1.0.0-prod .

docker build \
  --build-arg MYSCOUTEE_VERSION=1.0.0 \
  -f tools/docker/registry-edge.Dockerfile \
  -t myscoutee-registry-edge:1.0.0-prod \
  .

./tools/package-offline-stack.sh 1.0.0
```

```
bash tools/offline-stack-contract-test.sh
```

The packaging script does not build or pull. It requires both exact local images, supports `linux/amd64` only, refuses to overwrite an existing output artifact, rejects unexpected image IDs/platforms and common private-credential markers, and produces:

```
dist/myscoutee-registry-stack_1.0.0_linux_amd64.tar.gz
```

For identical image inputs and release metadata it creates a reproducible archive. The bundle contains:

- ☐ `images/myscoutee-registry.tar` and `images/myscoutee-registry-edge.tar`;
- ☐ a no-build/no-pull `compose.yaml` preserving read-only filesystems, capability restrictions, health checks, and the internal backend network;
- ☐ `registry.env.example`, `README.md`, `VERSION`, and an Nginx template audit copy;
- ☐ `IMAGE-MANIFEST.json` with stack/platform/source metadata, exact image IDs, archive paths, and archive SHA-256 values;
- ☐ `SHA256SUMS` covering every staged payload.

13.4 Remaining Release-Supply Controls

OPEN The bundle's checksums and image-manifest IDs detect damage and mismatch but do not authenticate MyScoutee as publisher. The current packager does not generate an OCI signature, detached archive signature, SBOM, vulnerability report, license pack, or release qualification record. The release authority must sign the outer archive through a separately trusted channel and deliver its verification key independently. Add SBOM, scan, compatibility, migration, and support evidence before treating the working-tree packager as a public release pipeline.

Do Not Put Secrets in an Image or Bundle

TLS keys, registry signing keys, VOPRF seeds, production environment files, SQLite data, partner credentials, and real announcement drafts are deployment state. Provision them at the target under the custody procedures in this manual.

14. Restricted and Mainland-China Operation

MyScoutee Registry can be placed inside a restricted national or private network so participating MyScoutee deployments do not cross a country firewall for registry traffic. The running Go registry has no required outbound protocol dependency; once images, DNS, TLS, and monitoring are locally available, requests remain between deployments and that registry origin.

14.1 Supported Sovereign Pattern

- Deploy the real registry and Nginx inside the target network with a locally reachable DNS name and a non-demo scope such as `partner-cn:mainland`.
- Keep the registry database, signing key, VOPRF keyring, TLS key, logs, and backups under the agreed regional custody and data-location policy.

- Register each in-region MyScoutee deployment directly to that one HTTPS origin, scope, and registry identity.
- Mirror or offline-deliver every build/runtime image and release artifact. Do not assume GitHub, Docker Hub, a public module proxy, public certificate automation, or a global monitoring endpoint is reachable or stable.
- Use a regional support and incident channel that remains available if cross-border connectivity is unavailable.

14.2 Using Both a Regional and a Global Registry

Separate deployment populations may use separate registries: for example, in-country MyScoutee deployments use the regional registry while other deployments use a global registry. Each registry remains a sovereign namespace with its own scope, keys, state, evidence, and partner authority.

A single MyScoutee deployment cannot send the same history to both, fail over between them, or merge their ledgers. Switching requires signed deactivation from the old registry and explicit registration to the new one. No bridge, relay, cross-scope synchronization, consolidated leaderboard, or shared settlement is implemented.

14.3 TLS and Private Networks

The current MyScoutee REAL client requires HTTPS. A publicly trusted certificate is simplest where available. A private CA can work only if every Java runtime is provisioned to trust that CA and the complete end-to-end package/container trust-store procedure is release-qualified. The registry repository does not ship that MyScoutee trust-store installer.

14.4 Partnership Control Checklist

Controlled Distribution in Version 1.0.0

The real registry is not publicly distributed as a supported release in version 1.0.0. Regional delivery therefore requires a controlled partnership, an authenticated artifact channel, and environment-specific qualification. Please contact MyScoutee if you want to deploy this topology.

- ☐ name the infrastructure, registry-administration, release, key-custody, backup, incident, and legal-review roles;
- ☐ approve the scope and publish the registry key fingerprint through an independent channel;
- ☐ define data location, backup export, retention, access review, and incident-notification rules;
- ☐ decide whether delivery uses a private regional image registry or signed offline media;
- ☐ keep registry release signing separate from registry protocol signing and partner host credentials;
- ☐ run install, power-loss, restore, upgrade, network-partition, and client retry drills entirely within the regional environment;
- ☐ obtain current local legal and regulatory review before launch; this technical manual is not a compliance determination.

Registry Data Is Reduced, Not Empty

The protocol excludes raw user profiles, Firebase UIDs, chat, access tokens, and payment transactions. It still stores company registered-address and verification-contact fields in protected SQLite tables, aggregate revenue, audit identifiers, and opaque global-identity commitments. Treat the volume and backups as sensitive regional data.

15. Security Operations and Incident Response

15.1 Preventive Controls

- least-privilege host/Docker access, MFA on infrastructure control planes, restricted management network, and audited local CLI use;
- mode-0600 key files, no symlinks, no production secrets in source/images, encrypted backups, and separate VOPRF/restricted-table access where practical;
- pinned release/image evidence, vulnerability monitoring, controlled TLS renewal, and reliable UTC time;
- external witnessing of registry identity, checkpoints, and selected Merkle heads;
- no manual SQLite UPDATE/DELETE, no nonce pruning, and no query-table repair scripts;
- reviewed idempotency keys and exact stale-target identifiers for local administrative decisions.

15.2 Incident Matrix

Incident	Immediate action	Recovery boundary
Integrity or health failure	Stop public ingress and writes; preserve container logs, host state, volume snapshot, image IDs, time evidence, and recent checkpoints.	Do not repair signed rows manually. Diagnose or restore a verified complete recovery unit.
SQLite/storage corruption	Stop both services and take a forensic snapshot before any tool touches the volume.	Restore database, signing key, and VOPRF keyring together. Validate every identity/head boundary.
Registry signing-key loss	Keep generation disabled and stop serving.	Restore the matching key. A new key is a new registry identity, not recovery.
Registry signing-key compromise	Isolate the host, preserve evidence, stop signing, and notify deployment operators through an independent channel.	v1 has no in-place key rotation. Establish a new independently commissioned registry and coordinate explicit client migration; preserve the compromised registry's evidence.
VOPRF keyring loss/compromise	Stop global-identity evaluation/link operations and restrict the protected tables.	Restore on loss. Rotation limits future use but does not undo exposure of old seeds/commitments or remove dictionary-test risk.
TLS/DNS compromise	Revoke/replace certificate or DNS credentials, isolate the edge, and compare the registry signing identity out of band.	Registry state may remain valid if the signing key was not exposed; verify before restoring traffic.
Clock rollback/drift	Stop signed writes and correct the approved time source.	Restart and verify; never edit accepted timestamps.

Incident	Immediate action	Recovery boundary
Disk full	Stop writes and expand or move storage through a qualified snapshot/restore.	Do not delete append-only history, durable nonces, Merkle nodes, or direct rows.
Private claim/backup disclosure	Revoke access, preserve access logs, identify copied records/backups, and follow local notification policy.	The private claim table is access-controlled but not field-level encrypted. Registry signatures do not remediate confidentiality loss.
Traffic flood / repeated 429	Use upstream firewall/rate controls and determine whether many valid deployments share one NAT address.	Change edge limits only after load and fairness testing; do not expose port 8080 as a bypass.

15.3 Registry Outage Effect

A registry outage pauses registry enrollment, QMAU/revenue delivery, announcements, network reads, and settlement history. MyScoutee product data remains local to each deployment; durable local outboxes can retry registry reports. After recovery, clients must use the original idempotency key and payload with fresh timestamp/nonce/signature rather than creating duplicate business records.

16. Retirement and Removal

16.1 Controlled Retirement

1. Freeze new enrollment and announce the retirement through a trusted channel.
2. Have every reachable MyScoutee deployment perform its signed disconnect/deactivation while the registry identity is still available. Preserve each deployment's archived receipts.
3. Finalize required checkpoints, export agreed evidence, take a complete final backup, and verify it on an isolated host.
4. Stop Nginx and the registry with `docker compose ... down` without `-v`.
5. Revoke or retire DNS/TLS credentials and remove public firewall rules.
6. Apply the approved retention/destruction policy to the production volume, provisioned key, VOPRF keyring, TLS key, environment, logs, images, and backups.

Volume Deletion Is Irreversible

`docker compose down -v` deletes the production volume containing the ledger and, in generated-key mode, both registry secrets. Do not run it as an uninstall shortcut. Resolve the exact volume, obtain written authorization, verify the final backup, and destroy only the intended target.

16.2 Removal Does Not Merge or Transfer History

Deleting a registry does not transfer its deployments, QMAU, revenue, claims, groups, settlements, or identity commitments to another registry. If clients were not deactivated before removal, their old binding cannot receive the signed deactivation receipt and requires a separately governed recovery/migration decision.

17. Known Gaps and Limitations

Gap	Current evidence	Required treatment
Human registry-identity confirmation is not fully enforced by the current product flow	The Go read-only identity endpoint and Java <code>inspect / confirm</code> endpoints exist. However, the current frontend registration action calls <code>register</code> directly, and Java's direct registration path performs a fresh cryptographic inspection but does not require a previously consumed inspection token. A source test explicitly permits selection without separate configuration.	Until fixed and qualified, require trusted endpoint catalog/TLS, expected scope, and an out-of-band key-ID/SPKI comparison. Do not claim that every enrollment has an enforced human key confirmation.
No supported/public installer release	The working tree includes a build-free, pull-free Linux/amd64 two-image offline bundle packager. Version 1.0.0 is not published as an authenticated supported installer, and no Debian/systemd installer or automatic registry updater exists.	Keep delivery controlled until the outer archive is authenticated, release qualification and support policy are complete, and the delivery channel is approved. Docker Engine and Docker Compose remain target prerequisites.
No OCI signature, SBOM publication, or immutable public image policy	No current repository tooling implements these release controls.	Use controlled partner delivery and record hashes/digests; implement supply-chain policy before broad publication.
No native backup/restore command	Scratch runtime contains no shell or SQLite utility; documentation requires quiescence, online backup API, or storage snapshot.	Use and qualify an external backup mechanism; record clean-host restore evidence.
Single-node SQLite	One serialized connection, one writable volume, no replication/failover protocol.	Define downtime/RPO/RTO and restore procedure. Do not invent active/active or cross-scope fallback.
No formal capacity/SLA matrix	Load, burst, topology, and production host drills remain external qualification items.	Benchmark expected deployment, QMAU, revenue, presence, announcement, proof, and health traffic before launch.

Gap	Current evidence	Required treatment
Health check cost	Health invokes integrity verification and Compose polls frequently.	Monitor latency/history growth; avoid extra high-frequency public polling; qualify any split liveness/readiness design.
Registry Ed25519 key rotation/HSM	No supported in-place registry identity rotation or HSM/threshold implementation.	Protect the key as a root trust asset; compromise requires controlled new-registry migration.
VOPRF single-operator threat	One operator can hold both VOPRF seeds and restricted commitments.	Separate and audit access. Threshold or separately governed/HSM-backed evaluation is future hardening.
Private claim fields are not field-level encrypted	Address and verification-contact fields are in protected append-only tables in the same SQLite database.	Restrict host/volume/CLI/backup access and apply retention/privacy controls.
No cross-registry federation	Scope is a strict signed domain; registries do not relay, synchronize, or consolidate.	Use separate populations and explicit deactivation/re-enrollment. Do not market multi-registry fallback as implemented.
Legal/payment boundary	Settlement and exit rails are technical evidence only.	Keep legal approval, payment, tax, contractual rounding, and beneficial-owner checks outside the protocol.

18. Quick Reference

Production Files

```
compose.production.yaml
compose.provisioned-key.yaml
.env.production.example
tools/docker/registry.conf.template
```

Offline Stack

```
tools/package-offline-stack.sh
tools/docker/offline/compose.yaml
tools/docker/offline/registry.env.example
dist/myscoutee-registry-
stack_1.0.0_linux_amd64.tar.gz
```

Persistent State

Logical volume: `registry-production-data`
 Database: `/data/registry.db`
 Generated key: `/data/registry-signing-key.pem`
 VOPRF: `/data/registry-global-identity-vopr-ke-yring.json`

Public Checks

GET `/healthz`
 GET `/v1/registry/identity`
 GET `/v1/announcements`
 HTTP → HTTPS 308 redirect

Never Expose

Container port 8080
 SQLite/WAL/SHM
 Registry/VOPRF/TLS private keys
 Production environment
 Private CLI output

Commissioning Checklist

- ☐ Real Compose, ordinary startup, non-demo scope, no demo seed.
- ☐ DNS and normal certificate-valid HTTPS verified externally.
- ☐ Port 8080 absent from host publications.
- ☐ Scope, key ID, SPKI fingerprint, image IDs/digests, and ledger head recorded.
- ☐ Generation switches disabled after first successful initialization.
- ☐ Complete encrypted backup and clean-host restore tested.
- ☐ MyScoutee operator compares the intended identity through an independent channel.
- ☐ Rate limits, NAT behavior, time sync, disk alerts, certificate expiry, and log rotation monitored.
- ☐ Open release drills and current identity-confirmation gap accepted by the accountable release authority.

Emergency Stop

```
docker compose \
  --env-file /etc/myscoutee-registry.env \
  -f compose.production.yaml \
  stop nginx registry
```

After stopping, preserve evidence and the exact volume before attempting recovery. Do not use `down -v`.

Companion Developer Manual

For wire formats, canonical messages, signatures, HTTP rules, idempotency, response contracts, and integration guidance, use the separate **MyScoutee Registry Protocol Developer Manual v1.0.0** (`MyScoutee_Registry_Protocol_Developer_Manual_v1.0.0_EN.pdf`). The individual Markdown files under `guides/manuals/protocols/` are controlled implementation evidence and PDF build inputs for maintainers; operators do not need to follow them one by one.